

# Tucupi: Implementação de um Analisador Léxico com JFlex e de um Analisador Sintático por Descida Recursiva

Jorge Vasconcelos, Lucas Salviano, Renan Abreu e Yuri Severo  
Disciplina de Autômatos e Compiladores  
Centro Universitário do Pará (CESUPA)  
Belém, PA, Brasil

**Resumo**—Este trabalho apresenta a evolução da Tucupi, uma linguagem imperativa experimental com palavras reservadas em português, desenvolvida na disciplina de Autômatos e Compiladores. Na etapa anterior foi construído o analisador léxico, gerado pelo JFlex 1.9.1. Nesta etapa, o analisador léxico foi mantido sem alterações e recebeu um analisador sintático escrito manualmente em Java pela técnica de descida recursiva, guiado por uma Gramática Livre de Contexto (GLC) LL(1) com 32 não terminais. O analisador sintático consome os tokens diretamente do scanner, constrói uma árvore sintática e, diante de uma entrada inválida, informa linha, coluna, token esperado, token encontrado e a produção em análise. O protótipo reconhece declarações, funções, condicionais, repetições, entrada e saída e expressões aritméticas e lógicas com dez níveis de precedência. Em 19 casos de teste automatizados (9 válidos e 10 inválidos), todos os resultados coincidiram com o esperado, inclusive a posição de cada erro. A análise semântica ainda não foi implementada.

**Index Terms**—analisador léxico, analisador sintático, JFlex, descida recursiva, gramática livre de contexto, árvore sintática

## I. INTRODUÇÃO

Um compilador costuma ser organizado em fases. As duas primeiras, a análise léxica e a análise sintática, formam a base do *front-end* [1]. A análise léxica lê caracteres e os agrupa em *tokens*; a análise sintática verifica se a sequência de tokens obedece à estrutura da linguagem, descrita por uma gramática livre de contexto [2]. Separar as duas fases simplifica cada uma: o analisador léxico trata de expressões regulares e autômatos finitos, enquanto o analisador sintático trata de produções e da estrutura hierárquica do programa.

A Tucupi é uma mini-linguagem criada pelo grupo com fins didáticos. Na etapa anterior, o trabalho se restringia à análise léxica: um *scanner* gerado pelo JFlex [7] transformava o código-fonte em uma lista de tokens. Ferramentas como Flex e JFlex, comparadas por Barbosa *et al.* no ensino de compiladores [8], cobrem apenas a fase léxica; a fase sintática precisa de outra solução. Nesta etapa ela foi escrita à mão, o que torna explícita a relação entre a gramática e o código. A etapa atual acrescenta, portanto, a análise sintática inicial:

Etapa anterior: fonte  $\rightarrow$  *lexer*  $\rightarrow$  tokens

Etapa atual: fonte  $\rightarrow$  *lexer*  $\rightarrow$  tokens  $\rightarrow$  *parser*  $\rightarrow$  árvore

As contribuições desta etapa são: (i) a manutenção do scanner JFlex existente, sem alterações na especificação; (ii) a definição de uma GLC para a Tucupi; (iii) um analisador sintático manual

Tabela I  
RECURSOS DA TUCUPI RECONHECIDOS PELO ANALISADOR SINTÁTICO

| Recurso                | Exemplo                                        |
|------------------------|------------------------------------------------|
| Declaração             | <code>var x : inteiro = 10;</code>             |
| Constante              | <code>constante LIMITE : inteiro = 100;</code> |
| Função                 | <code>funcao principal() -&gt; inteiro</code>  |
| Condicional            | <code>se (...) { ... } senao { ... }</code>    |
| Repetição              | <code>enquanto (n &gt; 0) { ... }</code>       |
| Repetição com contador | <code>para i = 1..10 passo 2 { ... }</code>    |
| Aritmética             | <code>x + y * 2, 2.0 ** 10</code>              |
| Comparação e lógica    | <code>x &gt; 10 &amp;&amp; ativo</code>        |
| Atribuição composta    | <code>soma += i &lt; 1;</code>                 |
| Chamada                | <code>dobro(dobro(3) + 1)</code>               |
| Índice e campo         | <code>dados[i], sensor.valor</code>            |
| Entrada e saída        | <code>leia(x); escreva("ok");</code>           |

por descida recursiva; (iv) a integração entre scanner e parser; (v) a construção e a impressão de uma árvore sintática; e (vi) o diagnóstico de erros sintáticos com posição no código-fonte. Das duas formas de implementação possíveis, o uso de ferramentas geradoras e a programação pura, o protótipo combina ambas: o scanner é gerado pelo JFlex e o parser é programação pura em Java. O código, os exemplos e as saídas citadas neste artigo estão no repositório do projeto<sup>1</sup>.

## II. A LINGUAGEM TUCUPI

A Tucupi é imperativa, tem palavras reservadas em português e cinco tipos primitivos: *inteiro*, *real*, *texto*, *caractere* e *logico*. Um programa começa com `programa Nome;`, segue com declarações globais e termina com uma lista de funções. Não há palavra reservada *main*: o bloco de execução principal é a função chamada *principal*, que o parser marca como ponto de entrada. A Tabela I resume os recursos reconhecidos pelo analisador sintático final; todos os exemplos foram aceitos pelo protótipo.

A Listagem 1 mostra o programa usado como exemplo ao longo do artigo e na demonstração. Ele contém uma declaração global, a função *principal*, uma expressão aritmética com precedência, uma regra lógica dentro de um *se/senao* e um retorno. Os números de linha são os do

<sup>1</sup><https://github.com/Yuri-Severo/tucupi-analisador-lexico>

Listagem 1. Programa de exemplo (linhas 10–24 do arquivo)

```

10 programa Demo;
11
12 var limite : inteiro = 10;
13
14 funcao principal() -> inteiro {
15     var x : inteiro = 5 + 2 * 3;
16
17     se (x > limite && x != 0) {
18         escreva("maior");
19     } senao {
20         escreva("menor");
21     }
22
23     retorne 0;
24 }

```

Tabela II

LEXEMA, CLASSE E TOKEN NO PROGRAMA DE EXEMPLO

| Lexema  | Classe              | Token             |
|---------|---------------------|-------------------|
| var     | palavra reservada   | PR_VAR            |
| limite  | identificador       | IDENTIFICADOR     |
| 10      | literal inteiro     | LIT_INTEIRO       |
| "maior" | literal de texto    | LIT_TEXTO         |
| +       | operador aritmético | OP MAIS           |
| &&      | operador lógico     | OP_E_LOGICO       |
| ;       | delimitador         | DEL_PONTO_VIRGULA |

arquivo exemplos/04\_sintatico\_valido.tcp, cujas linhas 1 a 9 são comentários.

### III. ANÁLISE LÉXICA

A análise léxica foi mantida da etapa anterior. Três conceitos orientam esta fase. O *lexema* é a sequência de caracteres reconhecida (por exemplo, *limite*). A *classe* é a categoria à qual o lexema pertence (identificador). O *token* é a unidade entregue ao parser e reúne o tipo, o lexema, o valor já convertido (quando existe) e a posição de início (linha e coluna).

O scanner é descrito no arquivo `tucupi.flex` por expressões regulares associadas a ações em Java. O JFlex converte essa especificação em um autômato finito determinístico e gera a classe `TucupiLexer`; na especificação atual, o autômato minimizado tem 212 estados. As diretivas `%line` e `%column` fazem o JFlex acompanhar a posição de cada token, e `%unicode` estende o alfabeto a todo o Unicode [7]. Para confirmar que a etapa léxica não mudou, o scanner foi gerado novamente com o JFlex 1.9.1: o arquivo resultante é idêntico ao versionado no repositório, e as saídas léxicas de referência continuam iguais.

A enumeração `TokenType` define 64 tipos de token: 19 palavras reservadas, 1 de identificador, 5 de literais, 27 operadores, 10 delimitadores e 2 especiais (EOF e ERRO). A Tabela II mostra exemplos retirados da Listagem 1.

Os identificadores são reconhecidos pela regra

```
Identificador = [:jletter:] [:jletterdigit:]*
```

em que as classes predefinidas `[:jletter:]` e `[:jletterdigit:]` correspondem aos métodos `Character.isJavaIdentifierStart` e

Listagem 2. Cadeia de tokens do programa de exemplo (saída real)

```

10 | PR_PROGRAMA IDENTIFICADOR(Demo) DEL_PONTO_VIRGULA
12 | PR_VAR IDENTIFICADOR(limite) DEL_DOIS_PONTOS
   ↳ PR_INTEIRO OP_ATRIBUI LIT_INTEIRO(10)
   ↳ DEL_PONTO_VIRGULA
14 | PR_FUNCAO IDENTIFICADOR(principal) DEL_ABRE_PAR
   ↳ DEL_FECHA_PAR OP_SETA PR_INTEIRO DEL_ABRE_CHA
15 | PR_VAR IDENTIFICADOR(x) DEL_DOIS_PONTOS PR_INTEIRO
   ↳ OP_ATRIBUI LIT_INTEIRO(5) OP MAIS LIT_INTEIRO(2)
   ↳ OP_VEZES LIT_INTEIRO(3) DEL_PONTO_VIRGULA
17 | PR_SE DEL_ABRE_PAR IDENTIFICADOR(x) OP_MAIOR
   ↳ IDENTIFICADOR(limite) OP_E_LOGICO IDENTIFICADOR(x)
   ↳ OP_DIFERENTE LIT_INTEIRO(0) DEL_FECHA_PAR
   ↳ DEL_ABRE_CHA
18 | PR_ESCREVA DEL_ABRE_PAR LIT_TEXTO("maior")
   ↳ DEL_FECHA_PAR DEL_PONTO_VIRGULA
19 | DEL_FECHA_CHA PR_SENAO DEL_ABRE_CHA
20 | PR_ESCREVA DEL_ABRE_PAR LIT_TEXTO("menor")
   ↳ DEL_FECHA_PAR DEL_PONTO_VIRGULA
21 | DEL_FECHA_CHA
23 | PR_RETORNE LIT_INTEIRO(0) DEL_PONTO_VIRGULA
24 | DEL_FECHA_CHA
   ↳ EOF (57 tokens, 0 erro(s) léxico(s))

```

`Character.isJavaIdentifierPart` do Java [7]. Com `%unicode`, nomes como *vazão\_média* e  $\sigma_{\text{amostral}}$  são identificadores válidos. As palavras reservadas aparecem antes dessa regra na especificação; como *var casa* com as duas regras com o mesmo comprimento, o JFlex escolhe a que aparece primeiro.

A saída da etapa léxica é a cadeia de tokens. A Listagem 2 mostra a cadeia completa do programa de exemplo, na saída real da opção `-tokens`, que agrupa os tokens pela linha do código-fonte em que começam (as quebras marcadas com `↳` foram inseridas para caber na coluna). A tabela completa de lexemas e tokens, com linha, coluna e valor de cada um dos 57 tokens, é gerada por `make tabela` e está em `docs/saidas/lexico_demo.txt`.

### IV. GRAMÁTICA LIVRE DE CONTEXTO

Uma GLC é formada por terminais, não terminais, um símbolo inicial e um conjunto de produções [2]. Na Tucupi, os *terminais* são exatamente os tipos de token produzidos pelo scanner, como `"se"`, `";"` e `IDENTIFICADOR`; a gramática não usa nenhum símbolo que o lexer não reconheça. Os *não terminais* são categorias estruturais, como `bloco`, `comando` e `expressao`. O *símbolo inicial* é `programa`. Cada *produção* indica como um não terminal pode ser escrito em função de terminais e de outros não terminais.

A gramática completa está no arquivo `docs/gramatica.ebnf`, em notação EBNF [6], e tem 32 não terminais. A Listagem 3 reproduz as produções centrais: `[ ]` indica parte opcional, `{ }` indica repetição e `|` separa alternativas.

A gramática foi escrita para ser LL(1) [3], [4]: em todo ponto de decisão, um único token de lookahead indica qual alternativa aplicar. Duas decisões de projeto garantem essa propriedade. Primeiro, atribuição e chamada de função começam com `IDENTIFICADOR`; a regra `comandoIdent` é a forma *fatorada à esquerda* das duas, e o token seguinte (`(` ou um operador de atribuição) decide o caminho. Segundo, o corpo de *se* é sempre um bloco entre chaves, o que elimina a

Listagem 3. Produções centrais da GLC (recorte de docs/gramatica.ebnf)

```

programa      = "programa", IDENTIFICADOR, ";";
               { declaracao }, { funcao }, EOF ;
declaracao    = ( "var" | "constante" ), IDENTIFICADOR,
               [ ":", tipo ], [ "=", expressao ], ";" ;
funcao        = "funcao", IDENTIFICADOR,
               "((", [ parametros ], ")",
               "->", tipo, bloco ;
bloco         = "{", { comando }, "}" ;
comando       = declaracao | condicional
               | enquanto | para
               | leitura, ";" | escrita, ";"
               | retorno, ";" | comandoIdent, ";" ;
comandoIdent  = IDENTIFICADOR,
               ( "(", [ argumentos ], ")"
               | { sufixo }, opAtribuicao, expressao ) ;
condicional   = "se", "(", expressao, ")", bloco,
               [ "senao", ( condicional | bloco ) ] ;

```

Tabela III  
RECURSOS SINTÁTICOS, PRODUÇÕES E EXEMPLOS

| Recurso       | Produção             | Exemplo                  |
|---------------|----------------------|--------------------------|
| Programa      | programa             | programa Demo;           |
| Declaração    | declaracao           | var x : real;            |
| Função        | funcao               | funcao f() -> real       |
| Bloco         | bloco                | { ... }                  |
| Condicional   | condicional          | se (c) {...} senao {...} |
| Repetição     | enquanto, para       | enquanto (c) {...}       |
| Atribuição    | comandoIdent         | x += 1;                  |
| Chamada       | comandoIdent         | dobro(y);                |
| Entrada/saída | leitura, escrita     | leia(x);                 |
| Expressão     | expressao...primario | 5 + 2 * 3                |

ambiguidade do “senão pendente”: cada *senao* pertence ao se imediatamente anterior, e a construção *senao* se surge da própria regra condicional. A Tabela III relaciona os recursos sintáticos às produções que os definem.

As expressões ocupam um não terminal por nível de precedência, do menos para o mais prioritário:

```

expressao      = expressaoOu ;
expressaoOu    = expressaoE, { "||", expressaoE } ;
expressaoE     = igualdade, { "&&", igualdade } ;
igualdade      = relacional, { "==" | "!=" }, relacional ;
relacional     = deslocamento,
               { "<" | "<=" | ">" | ">=" }, deslocamento ;
deslocamento  = soma, { "<<" | ">>" }, soma ;
soma           = produto, { "+" | "-" }, produto ;
produto        = unario, { "*" | "/" | "%" }, unario ;
unario         = ("!" | "+" | "-"), unario | potencia ;
potencia       = posfixo, [ "**", unario ] ;
posfixo        = primario, { sufixo } ;

```

Como produto fica abaixo de soma, a multiplicação é agrupada antes da adição. A repetição { } substitui a recursão à esquerda, que um parser descendente não trata, e produz associatividade à esquerda ( $a - b - c$  equivale a  $(a - b) - c$ ). A potência é a única regra recursiva à direita, o que a torna associativa à direita.

## V. ANALISADOR SINTÁTICO

O analisador sintático (*AnalisadorSintatico.java*) foi implementado manualmente em Java pela técnica de *descida recursiva* [1], [5]: cada não terminal da gramática corresponde a um método, e a pilha de chamadas do Java acompanha a derivação. Não foi usado nenhum gerador de parser. O analisador mantém um único token de *lookahead*, isto é, o

Listagem 4. Método *esperar* (código real)

```

private Token esperar(TokenType tipo, String mensagem) {
    if (!verificar(tipo)) {
        throw erro(mensagem, descrever(tipo));
    }
    Token t = atual;
    if (tipo != TokenType.EOF) {
        avancar();
    }
    return t;
}

```

próximo token ainda não consumido, e decide qual produção aplicar olhando apenas para ele.

Quatro métodos auxiliares traduzem a EBNF para o código: *avancar()* consome o token atual e pede o próximo ao scanner; *verificar(t)* testa o tipo do token atual sem consumi-lo; *aceitar(t)* consome o token se ele for do tipo *t*; e *esperar(t)* exige um token do tipo *t*. Com eles, um terminal da gramática vira uma chamada a *esperar*, uma parte opcional [ *X* ] vira um *if*, uma repetição { *X* } vira um *while* e uma alternativa vira um *switch* sobre o token atual. A Listagem 4 mostra o método *esperar*, que concentra a detecção de erros.

Os níveis binários de expressão compartilham um único esquema,  $X = Y \{ \text{op } Y \}$ , implementado pelo método *binario*. Ele recebe o método do nível seguinte e os operadores do nível atual; a cada operador encontrado, cria um nó cujo filho esquerdo é a árvore construída até então. Assim, cada nível tem um método de uma linha, por exemplo *soma()* chama *binario(this::produto, OP MAIS, OP MENOS)*, e a precedência resulta da ordem das chamadas.

A Tabela IV rastreia a análise da linha 15 da Listagem 1, *var x : inteiro = 5 + 2 \* 3;*, a partir do momento em que *comando()* encontra o token *PR\_VAR*. As etapas seguem o código do parser; os níveis intermediários entre *expressao* e *primario* foram abreviados com “→ ... →”.

Quando o token atual não serve para nenhuma alternativa, o analisador lança a exceção *ErroSintatico*. Ela registra a mensagem, o token encontrado, o que era esperado, o último token aceito e a pilha de produções ativas (por exemplo, *programa > funcao > bloco > comando > declaracao*). O último token aceito é informado porque, quando falta um ;, o erro só é percebido no token seguinte, que pode estar em outra linha. Nesta etapa o analisador para no primeiro erro sintático e não faz recuperação.

### A. Decisões de implementação

Três decisões simplificaram a entrega. Primeiro, o parser foi escrito à mão em vez de gerado por uma ferramenta como o CUP, para que a correspondência entre cada produção e cada método ficasse explícita e o scanner JFlex pudesse ser reaproveitado sem adaptações. Segundo, a árvore usa uma única classe de nó genérica, evitando dezenas de classes de AST numa etapa que ainda não tem análise semântica. Terceiro, a demonstração e os testes funcionam sem acesso à internet: a interface de linha de comando (*MainSintatico*) usa o scanner já gerado, e o alvo *make demo* executa o caso válido

Tabela IV  
RASTREAMENTO DA LINHA 15 PELO PARSE

| Lookahead         | Método                     | Ação                         |
|-------------------|----------------------------|------------------------------|
| PR_VAR            | comando                    | escolhe declaracao()         |
| PR_VAR, x, :      | declaracao                 | consome var, x, :            |
| PR_INTEIRO        | tipo                       | consome inteiro              |
| OP_ATRIBUI        | declaracao                 | consome =; chama expressao() |
| LIT_INTEIRO(5)    | expressao → ... → primario | consome 5: LITERAL 5         |
| OP_MAIS           | produto                    | não é * / %; devolve 5       |
| OP_MAIS           | soma                       | consome +; chama produto()   |
| LIT_INTEIRO(2)    | primario                   | consome 2                    |
| OP_VEZES          | produto                    | consome *; lê 3              |
| DEL_PONTO_VIRGULA | produto                    | cria MULTIPLICACAO(2, 3)     |
| DEL_PONTO_VIRGULA | soma                       | cria SOMA(5, MULTIPLICACAO)  |
| DEL_PONTO_VIRGULA | declaracao                 | esperar(;); cria DECL_VAR    |

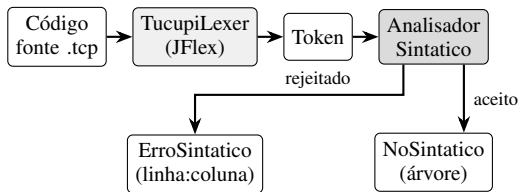


Figura 1. Arquitetura do protótipo: o parser pede um token por vez ao scanner e produz uma árvore ou um erro.

e o inválido. A opção `-tokens` mostra a cadeia de tokens antes da árvore, e a opção `-ascii` desenha a árvore apenas com caracteres ASCII.

## VI. INTEGRAÇÃO ENTRE SCANNER E PARSE

A Fig. 1 mostra a arquitetura. O JFlex trabalha sobre caracteres e produz tokens; o parser trabalha apenas sobre tokens e nunca lê o arquivo. O parser recebe o TucupiLexer no construtor e, a cada chamada de `avancar()`, pede o próximo token pelo método `proximoToken()`, o mesmo que a etapa léxica já oferecia. Nenhuma regra léxica foi duplicada no parser. O trecho abaixo, retirado de `MainSintatico.java`, é tudo o que a integração exige:

```
TucupiLexer lexer = new TucupiLexer(new
    ↳ StringReader(fonte));
parser = new AnalisadorSintatico(lexer);
raiz = parser.programa();
```

As responsabilidades pelos erros ficam separadas. Erros léxicos, como um caractere fora do alfabeto ou um literal inteiro fora da faixa de 32 bits, continuam sendo registrados pelo scanner, que não interrompe a varredura. Erros sintáticos, como um `; ausente`, são detectados pelo parser. Quando o scanner devolve um token `ERRO`, o parser interrompe a análise nesse ponto, e a interface de linha de comando lista os erros léxicos registrados. No arquivo `02_errores.tcp` da etapa anterior, por exemplo, a análise para em 12:23, no caractere `@`, que é um erro léxico. A Tabela V compara os dois tipos de erro com exemplos do repositório.

Tabela V  
ERRO LÉXICO E ERRO SINTÁTICO NO PROTÓTIPO

|                   | Erro léxico      | Erro sintático        |
|-------------------|------------------|-----------------------|
| Detectado por     | TucupiLexer      | AnalisadorSintatico   |
| Unidade analisada | caracteres       | tokens                |
| Exemplo           | 25 @ 3 (12:23)   | ; ausente (6:5)       |
| Arquivo           | 02_errores.tcp   | 05_sintatico_erro.tcp |
| Registro          | ErroLexico       | ErroSintatico         |
| Comportamento     | scanner continua | parser para           |

## VII. ÁRVORE SINTÁTICA

Cada produção reconhecida gera um nó da classe `NoSintatico`. Para não criar uma classe por construção da linguagem, todo nó tem apenas um *tipo* (`PROGRAMA`, `SE`, `SOMA`...), um *valor* opcional (nome, operador ou lexema) e uma lista de *filhos*. Parênteses não geram nós, porque o agrupamento já está expresso na forma da árvore. A classe `ImpressoraArvore` imprime o resultado no estilo do comando `tree`.

A Fig. 2 é a saída real do protótipo para a Listagem 1, obtida com a opção `-ascii`. A declaração da linha 15 mostra a precedência: a `MULTIPLICACAO` fica abaixo da `SOMA`, ou seja, `5 + 2 * 3` foi reconhecido como `5 + (2 * 3)`. Na condição do `se`, o operador `&&` fica acima das comparações, por ter precedência menor.

## VIII. RESULTADOS E AVALIAÇÃO

### A. Caso 1: programa válido

Para a Listagem 1, o scanner produziu 57 tokens sem erros léxicos (Listagem 2), o parser consumiu os 57 tokens e construiu a árvore da Fig. 2, com 29 nós. O protótipo informou:

```
Tokens consumidos pelo parser: 57
Nós na árvore sintática.....: 29
Ponto de entrada.....: função principal, linha 14
Resultado: ACEITO -- a entrada pertence à linguagem gerada
↳ pela GLC.
```

Ou seja, análise léxica sem erros e análise sintática aceita.

### B. Caso 2: programa inválido

O arquivo `05_sintatico_erro.tcp` contém um programa sem o `; ao fim` da declaração da linha 4, seguido por um `se` na linha 6. A Listagem 5 traz a saída real.

```

PROGRAMA Demo
|-- DECL_VAR limite : inteiro
|   '-- LITERAL 10
|-- FUNCAO principal -> inteiro [ponto de entrada]
|   '-- BLOCO
|       |-- DECL_VAR x : inteiro
|       |   '-- SOMA +
|       |       |-- LITERAL 5
|       |       '-- MULTIPLICACAO *
|       |           |-- LITERAL 2
|       |           '-- LITERAL 3
|       |-- SE
|       |   |-- CONDICAO
|       |   |   '-- E &&
|       |   |       |-- MAIOR >
|       |   |       |   |-- IDENT x
|       |   |       |   '-- IDENT limite
|       |   |       '-- DIFERENTE !=
|       |   |           |-- IDENT x
|       |   |           '-- LITERAL 0
|       |   |-- BLOCO
|       |   |   '-- ESCREVA
|       |   |       '-- LITERAL "maior"
|       |   '-- SENAO
|       |       |-- BLOCO
|       |       |   '-- ESCREVA
|       |       |       '-- LITERAL "menor"
|       '-- RETORNE
|           '-- LITERAL 0

```

Figura 2. Árvore sintática da Listagem 1 (saída real, 29 nós).

#### Listagem 5. Diagnóstico do caso inválido (saída real)

```

Erro sintático em 6:5
Mensagem... falta ";" no fim da declaração
Esperado... DEL_PONTO_VIRGULA (";")
Encontrado: PR_SE ("se")
Após..... LIT_INTEIRO ("10") em 4:23
Produções.. programa > funcao > bloco > comando > declaracao

4 |      var x : inteiro = 10
  |                               ^ após este ponto
  |
  |
6 |      se (x > 2) {
  |          ^ erro detectado aqui

```

Todos os lexemas do arquivo são válidos: o scanner reconheceu 33 tokens sem nenhum erro léxico, e por isso não havia motivo para rejeitar a entrada na primeira fase. O problema está na ordem dos tokens: a produção `declaracao` exige ";" depois da expressão, mas o token seguinte é `PR_SE`. O parser só percebe a violação ao encontrar o `se` (6:5) e, por isso, aponta também o último token aceito (10, em 4:23), que é onde o ; deveria estar.

#### C. Casos de teste

O repositório inclui um executor de testes sem dependências externas (`TestesSintaticos.java`, acionado por `make testes`). Os casos válidos devem ser aceitos; no caso de precedência, a árvore impressa é comparada com uma árvore esperada gravada em arquivo. Os casos inválidos devem ser rejeitados exatamente na linha e coluna anotadas no próprio arquivo. A Tabela VI resume a execução.

#### D. Resultados quantitativos

A Tabela VII reúne apenas valores medidos no código final. Além dos testes, os dois programas de exemplo da etapa léxica que não contêm erros também foram aceitos pelo parser:

Tabela VI  
CASOS DE TESTE DO ANALISADOR SINTÁTICO

| Caso                          | Esperado     | Resultado         |
|-------------------------------|--------------|-------------------|
| Declaração simples            | aceito       | aceito            |
| Precedência (árvore)          | aceito       | aceito, árvore ok |
| se                            | aceito       | aceito            |
| se/senao se/senao             | aceito       | aceito            |
| enquanto                      | aceito       | aceito            |
| Funções                       | aceito       | aceito            |
| Chamadas                      | aceito       | aceito            |
| Só a função principal         | aceito       | aceito            |
| para, +=, «, índice, campo    | aceito       | aceito            |
| ; ausente                     | erro em 5:5  | erro em 5:5       |
| ) ausente                     | erro em 4:15 | erro em 4:15      |
| ) ausente                     | erro em 6:1  | erro em 6:1       |
| 1 + * 2                       | erro em 4:27 | erro em 4:27      |
| retorne 0 0;                  | erro em 4:15 | erro em 4:15      |
| senao sem se                  | erro em 4:5  | erro em 4:5       |
| Declaração global após função | erro em 4:1  | erro em 4:1       |
| Função sem -> tipo            | erro em 3:20 | erro em 3:20      |
| Comparação como comando       | erro em 4:7  | erro em 4:7       |
| Arquivo sem programa Nome;    | erro em 2:1  | erro em 2:1       |

Tabela VII  
MÉTRICAS DO PROTÓTIPO (MEDIDAS NO CÓDIGO FINAL)

| Métrica                                      | Valor            |
|----------------------------------------------|------------------|
| Tipos de token (TokenType)                   | 64               |
| Estados do AFD minimizado (JFlex)            | 212              |
| Não terminais da GLC                         | 32               |
| Níveis de precedência de expressões          | 10               |
| Métodos de produção no parser                | 30               |
| Casos de teste válidos / inválidos           | 9 / 10           |
| Casos com resultado igual ao esperado        | 19 de 19         |
| Exemplos da etapa léxica aceitos pelo parser | 2 de 2 sem erros |

`01_valido.tcp` (335 tokens, árvore com 169 nós), que exercita `para`, `enquanto`, `**`, `«` e atribuições compostas, e `03_unicode.tcp` (159 tokens, 64 nós), com identificadores acentuados.

## IX. LIMITAÇÕES

O protótipo cobre apenas as duas primeiras fases de um compilador, e o escopo desta etapa tem limites claros:

- não há análise semântica: tipos, declaração antes do uso, compatibilidade do retorno e a existência da função principal não são verificados (a ausência de principal gera apenas um aviso);
- não há geração de código nem execução de programas;
- o parser para no primeiro erro sintático, sem recuperação;
- a palavra reservada `ate` é reconhecida pelo scanner, mas a gramática usa apenas `..` no `para`;
- acesso por índice e por campo é aceito nas expressões, mas a linguagem ainda não tem sintaxe para declarar vetores ou registros;
- os testes verificam aceitação, rejeição e posição do erro, mas não comparam todas as árvores, apenas a do caso de precedência.

## X. CONCLUSÃO

O protótipo da Tucupi passou a contemplar as duas primeiras fases clássicas de um compilador. O scanner gerado pelo JFlex foi mantido sem alterações e integrado, pelo método `proximoToken()`, a um analisador sintático manual por descida recursiva. A GLC LL(1) define o subconjunto reconhecido e a precedência das expressões; o parser constrói uma árvore sintática que torna essa estrutura visível e, diante de uma entrada inválida, informa a posição do erro, o que era esperado e o que foi encontrado. Os 19 casos de teste e os exemplos das duas etapas apresentaram os resultados esperados.

Como trabalhos futuros, ficam a análise semântica (tabela de símbolos com escopos e verificação de tipos), a recuperação de erros sintáticos, a expansão da gramática (vetores, registros e o uso de `ate`), a interpretação dos programas e a geração de código intermediário.

## REFERÊNCIAS

- [1] A. V. Aho, M. S. Lam, R. Sethi e J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, EUA: Addison-Wesley, 2006.
- [2] N. Chomsky, "Three models for the description of language," *IRE Trans. Inf. Theory*, vol. 2, no. 3, pp. 113–124, 1956.
- [3] P. M. Lewis II e R. E. Stearns, "Syntax-directed transduction," *J. ACM*, vol. 15, no. 3, pp. 465–488, jul. 1968.
- [4] D. E. Knuth, "Top-down syntax analysis," *Acta Informatica*, vol. 1, pp. 79–110, 1971.
- [5] N. Wirth, *Compiler Construction*. Harlow, Reino Unido: Addison-Wesley, 1996.
- [6] *Information technology — Syntactic metalanguage — Extended BNF*, ISO/IEC 14977:1996, 1996.
- [7] JFlex, "JFlex User's Manual," versão 1.9.1. [Online]. Disponível: <https://jflex.de/manual.html>
- [8] C. Barbosa, R. Bonidia e J. Neto, "Flex, JFlex e GALS: Ferramentas de apoio ao ensino de compiladores," em *Anais do XXVII Workshop sobre Educação em Computação (WEI)*, Belém, 2019, pp. 176–187, doi: 10.5753/wei.2019.6628.